ACTIVITAT
Objectius: ■ Processat de dades amb JSON en MongoDB
Instruccions: ■ Resoleu els exercicis proposats i pengeu el repositori a GitHub
Criteris d'avaluació: ■ Es valorarà la presentació i els comentaris al codi
Entrega: ■ A Moodle enllaç a repositori GitHub. - A dins del repositori, en un directori anomenat "doc" inclout aquest document completat en format pdf i amb el nom "memoria.pdf"
Repositori d'exemple: ■ https://github.com/jpala4-ieti/DAM-M0486-Tema4-RA5-PR3.2-Punt-Partida-25-26

Exercicis

Exercici 1. Mini-Servidor WebSocket de Registre d'Esdeveniments de Joc amb Client Node.js (10 punts)

Crear un servidor WebSocket bàsic en Node.js que escolti connexions entrants. El servidor rebrà missatges en format JSON simulant el moviment 2D d'un jugador en una partida des d'un client Node.js i els emmagatzemarà en una col·lecció de MongoDB.

Tecnologies a utilitzar:

- Node.js
- **Biblioteca ws:** Per implementar WebSockets tant al servidor com al client (npm install ws).
- MongoDB (utilitzant la mateixa instància de Docker que a la PR3.1)
- Driver de MongoDB (mongodb).

Què cal fer?

- Crea una fitxa de requisits per la funcionalitat que es demana resoldre. Parteix de la informació de l'annex i amb ajuda de la IA Claude, indica quins altres aspectes poden millorar la fitxa de requisits **(2.5 punts)**
- Crea una carpeta anomenada pr32_websocket_link
- Crea dos fitxers: websocket_server.js i websocket_client.js cal implementar aquesta funcionalitat **(5.5 punts)**:
    - El jugador s'ha de moure amb les fletxes.
    - El client enviarà la informació al servidor.
    - Sempre i quan el jugador estigui en moviment la informació de posició es considerarà la mateixa partida. Cada moviment en guardarà a MongoDB (1 document per moviment, però amb informació per associar-lo a la partida).
- Si el jugador passa 10 segons sense moure's es considera finalitzada la partida, el calcula la distància en línia recta entre el punt inicial i final i s'informa al client **(2 punts)**
- En els servidor cal implementar logs (pantalla i fitxer) amb winston.

Ampliació d'Infraestructura (Grafana Loki)

1. L'entorn s'ha d'ampliar amb Grafana Loki (que actuarà com a base de dades especialitzada en logs) i Grafana (que proporcionarà la interfície visual per fer cerques i panells).
2. **Integració Node.js / Winston**: S'ha de modificar la configuració del servidor WebSocket per enviar les dades a Loki. Això s'implementarà afegint un nou *transport* directe a winston (ex: utilitzant la llibreria `winston-loki`) o configurant un contenidor agent (Promtail) que llegeixi l'arxiu físic de logs i l'enviï a Loki.

Annex

Ficha de requisitos PR3.2

Qué nos piden
Crear un servidor WebSocket básico en Node.js, que escuche conexiones entrantes. El servidor recibirá mensajes en formato JSON simulando el movimiento 2D de un jugador en una partida desde un cliente Node.js, y los almacenará en una colección de MongoDB.

Ficha de requisitos: Sistema de cálculo de movimiento

Autor: Rubén Bellido **Fecha:** 2026-04-12

1. Objetivo Principal

Desarrollar dos programas con Node.js: Un cliente y un servidor.

El cliente debe enviar al servidor el input de las flechas del teclado para simular un movimiento que calculará el servidor. Para ello le enviará un JSON con un formato concreto si el cliente se está moviendo.

El servidor recibirá los inputs de movimiento del cliente cada vez que haga un movimiento diferente. Los movimientos se almacenan en MongoDB, con información que identifique la partida actual. Si el cliente no se mueve durante 10s, se calcula cuánto se ha movido (línea recta entre punto final e inicial) y envía información al cliente.

2. Requisitos Funcionales (Qué debe hacer)

- **RF-01**: El servidor WebSocket debe iniciarse y escuchar conexiones entrantes en un puerto configurable.
- **RF-02**: El cliente WebSocket debe poder conectarse al servidor.
- **RF-03**: El servidor debe generar un identificador para la partida actual.
- **RF-04**: El cliente debe enviar un JSON al servidor cada vez que cambie su movimiento utilizando las flechas del teclado.
- **RF-05**: El formato JSON del cliente debe incluir: direction (string), timestamp (number).
- **RF-06**: El servidor ha de validar la estructura de los mensajes recibidos. Si no son válidos, los descarta.
- **RF-07**: El servidor ha de añadir un timestamp (number) a cada lectura válida recibida.
- **RF-08**: El servidor debe almacenar cada lectura válida en una colección de MongoDB. Cada entrada tendrá como claves: gameId (string), direction (string), timestampClient (number), timestampProcessed (number).
- **RF-09**: El servidor ha de comprobar, en caso de que el cliente no se esté moviendo, cuánto tiempo ha pasado desde que se movió por última vez. Si han pasado más de 10s, el servidor da por finalizada la partida y consolida el resultado.
- **RF-10**: El servidor calcula cuánto se ha desplazado el cliente a partir de los registros almacenados en MongoDB. Para ello, calcula el desplazamiento entre cada cambio de movimiento a partir del timestamp del cliente y una velocidad. Cuando tiene la posición final definida, calcula la distancia al punto inicial en una línea recta calculando el módulo del vector de posición.
- **RF-11**: El servidor envía al cliente la información de la partida en un JSON con este formato: event='FINISHED', totalDistance (float).
- **RF-12**: El cliente imprime por pantalla la información de la partida recibida por parte del servidor.

3. Requisitos No Funcionales (Cómo lo debe hacer)

- **RNF-01 (Logging):** El servidor ha de implementar logs con `winston`. Se han de registrar los sucesos clave (arranque, conexión/desconexión, error, mensajes recibidos, validación de esos mensajes recibidos, dato almacenado, resultado calculado, mensaje enviado a cliente) en la consola y en un fichero (`server.log`).
- **RNF-02 (Sostenibilidad):** El código debe estar comentado y seguir buenas prácticas para que sea fácil de entender y modificar. La configuración esencial (puerto del servidor, URI de conexión con MongoDB, velocidad de movimiento, tiempo sin moverse para acabar la partida) ha de ser fácilmente modificable (un .env, un .xml de configuración, etc.).
- **RNF-03 (Fiabilidad Básica):** El servidor ha de poder gestionar errores comunes (error de conexión con la BBDD, con el cliente, etc.) registrando el error sin detenerse completamente, si es posible.

4. Formato Mensaje JSON (Cliente -> Servidor)

```
{
   "direction": "UP",     // Estoy apretando la flecha "Arriba"
   "timestamp": 1712234567890  // Timestamp en ms generado con Date.now()
}
```

5. Formato mensaje JSON (Servidor -> Cliente)

```
{
   "totalDistance": 73.4,          // Cuánto me he movido desde el punto
inicial
   "gameStartTime": 1712234567890, // Fecha comienzo partida
   "gameEndTime": 1712234568000,   // Fecha final partida
}
```